

DESIGN AND TEST SPECIFICATION - SDP PHASE III

AI-Powered Navigator for Ethical Obscure Net Exploration and Intelligence Gathering

Complete project documentation for **DarkPulse**, a defensive OSINT and threat-intelligence console that collects, normalizes, searches, visualizes, and evaluates cyber-risk signals from multiple approved sources.



ADVISOR
Hafiz Bilal Shahid

TEAM
#F25CS064

DOCUMENT TYPE
Complete Technical and Product Report

PREPARED FOR
Final Year Project Evaluation and Demonstration

Member Name	Primary Responsibility
Ali Naeem	Database, Research, and Testing
M. Omar Zahid	Frontend Development and Dashboard Design
M. Abdullah	Backend Development and AI Integration

This document is written in English and explains the product, audience, architecture, implementation choices, modules, evaluation approach, and presentation strategy.

Table of Contents

- | | |
|------------------------------------|---|
| 1. Executive Summary | 9. Frontend Choice and Design Rationale |
| 2. Problem Statement | 10. Feature-by-Feature Explanation |
| 3. Target Audience and User Value | 11. AI, Classifiers, Confidence, and Accuracy |
| 4. Why DarkPulse Is Unique | 12. Security, Ethics, and Privacy Controls |
| 5. Team Roles and Responsibilities | 13. Testing Strategy |
| 6. System Architecture | 14. How to Present DarkPulse to the Audience |
| 7. Technology Stack and Rationale | 15. Limitations and Future Work |
| 8. MongoDB vs SQL Justification | 16. Conclusion |

1. Executive Summary

DarkPulse is a modular cyber-intelligence platform designed for ethical, defensive monitoring. It gathers intelligence from security news, exploit advisories, leak sources, defacement records, social/forum monitoring, GitHub repository scans, APK and PC software checks, credential exposure datasets, and sensitive-document analysis workflows.

The core idea is simple: instead of forcing analysts to run many isolated scripts and manually inspect raw JSON files, DarkPulse brings collection, storage, filtering, semantic search, dashboard visualization, and evidence review into one local intelligence console.

Product promise: DarkPulse helps authorized users move from scattered threat data to structured, searchable, explainable intelligence.

2. Problem Statement

Cybersecurity teams, students, and researchers often monitor many places at once: news sites, exploit databases, defacement mirrors, leak pages, social platforms, GitHub repositories, software marketplaces, and local exposure dumps. Without a unified tool, this creates duplicated effort, inconsistent formats, missed signals, and weak reporting.

Traditional manual workflows are slow because every source has its own structure. One source may publish a title and URL; another may publish actor names, countries, screenshots, and leak dates; another may only provide raw text. DarkPulse addresses this by normalizing heterogeneous evidence into MongoDB and making it accessible through a single dashboard.

3. Target Audience and User Value

SOC Analysts

They can monitor active leaks, exploits, defacements, and suspicious mentions from one dashboard, reducing manual source switching.

Cybersecurity Researchers

They can collect structured evidence, run semantic searches, inspect raw context, and compare signals across categories.

Educational Institutions

Students and faculty can demonstrate OSINT concepts, data normalization, secure scanning, and AI-supported search in a controlled local environment.

Small Organizations

Teams without expensive threat-intelligence platforms can use DarkPulse as a local defensive monitoring and reporting console.

The audience will be impressed because the project is not only a static dashboard. It performs real workflows: scan now, stop scan, live feed refresh, semantic search, repository vulnerability analysis, exposure search, translation, SEO analysis, and crawler-health monitoring.

4. Why DarkPulse Is Unique

- **Unified intelligence streams:** news, leaks, exploits, defacement, social/forum signals, API tools, APK checks, PC software checks, credentials, and confidential data analysis are visible in one product.
- **One-click intelligence update:** the dashboard can start a hidden fast background scan while old cached records remain visible.
- **Mongo-backed normalization:** data from many source formats is upserted, deduplicated, and transformed into consistent dashboard items.
- **Semantic and natural-language search:** users can search intelligence using normal language instead of only exact keywords.
- **Self-healing monitor:** the system checks collector scripts, detects HTML drift, measures selector health, stores snapshots, and can generate repair previews.
- **Defensive scanning modules:** repository, APK, PC-game, SEO, credential, and confidential-file workflows extend the product beyond passive feed viewing.
- **Local-first architecture:** MongoDB, FastAPI, and local files allow practical use without depending completely on paid external platforms.

5. Team Roles and Responsibilities

5.1 Ali Naeem - Database, Research, and Testing

Ali's role focuses on the data foundation of DarkPulse. This includes researching threat-intelligence sources, understanding how each source structures its information, designing persistence collections, checking deduplication behavior, and validating whether records are stored correctly in MongoDB.

- MongoDB collection planning for articles, raw items, entities, source status, credentials, repository scans, APK scans, and analysis results.
- Testing data ingestion from different collector categories.
- Verifying that dashboard counts and feed results match stored database records.
- Preparing test cases for confidential-data detection, masking expectations, and result rendering.

5.2 M. Omar Zahid - Frontend Development and Dashboard Design

Omar's role covers the user-facing console. The frontend is responsible for making complex intelligence understandable through navigation, cards, tables, filters, maps, reports, modals, scan states, progress indicators, translation controls, and export-ready result views.

- Designing the DarkPulse command center, sidebar navigation, live feed, and feature screens.
- Implementing scan buttons, pagination, search inputs, modals, and report views in JavaScript.
- Making every module easy to demonstrate: Scan Now, semantic search, credential checker, repository scan, SEO, APK, PC game, and healing monitor.
- Maintaining a professional cybersecurity visual style that feels like an intelligence console rather than a simple CRUD website.

5.3 M. Abdullah - Backend Development and AI Integration

Abdullah's role covers FastAPI services, authentication, scan orchestration, MongoDB queries, AI-supported workflows, translation, natural-language querying, semantic search, scanner integration, and API endpoints used by the frontend.

- Building backend endpoints in `ui_server.py` using FastAPI and asynchronous MongoDB access through Motor.
- Integrating the orchestrator and background intelligence update workflow.
- Connecting AI tools such as Gemini-backed translation/NLQ and Ollama-backed local Mongo summarization.
- Implementing scanners for repository security, APK mirrors, PC-game sources, SEO posture, credentials, and confidential-data indicators.

6. System Architecture

DarkPulse follows a layered architecture. The browser loads `index.html`, `style.css`, and `app.js`. The frontend talks to the FastAPI backend in `ui_server.py`. The backend reads/writes MongoDB and starts collector/scanner workflows. Collector scripts normalize data using shared data models and persistence helpers.

Layer	Main Components	Purpose
Presentation	<code>index.html</code> , <code>style.css</code> , <code>app.js</code>	Dashboard, navigation, filters, cards, tables, modals, scan controls, live feed, reports.
API and Orchestration	<code>ui_server.py</code> , <code>orchestrator.py</code>	REST endpoints, authentication, scan jobs, cache, background tasks, scanner calls.
Data Persistence	<code>mongo_persistence.py</code> , MongoDB	Raw records, normalized records, entities, scan history, user data, source status, analysis results.
Collectors	<code>news_collector</code> , <code>leak_collector</code> , <code>exploit_collector</code> , <code>defacement_collector</code> , <code>social_collector</code> , <code>api_collector</code>	Gather data from approved OSINT and scanner sources.
Operations	<code>healing</code> , logs, snapshots	Monitor collector health, detect drift, store snapshots, generate repair previews.

7. Technology Stack and Rationale

Technology	Used For	Why It Was Selected
Python	Backend, collectors, scanner logic	Strong ecosystem for scraping, automation, security tooling, async APIs, and data processing.
FastAPI	HTTP API server	Fast asynchronous API framework with clean endpoint definitions, validation support, and easy Uvicorn deployment.
Motor / PyMongo	MongoDB access	Motor supports async backend routes; PyMongo is useful for synchronous collector persistence.
MongoDB	Primary database	Flexible document model fits inconsistent OSINT data better than rigid relational tables.
Vanilla HTML/CSS/JS	Frontend dashboard	Simple deployment, no heavy build step, direct control over UI behavior, and easy local hosting.
BeautifulSoup / Requests	HTML parsing and HTTP collection	Reliable tools for lightweight source parsing and local SEO auditing.
Playwright	Browser-driven checks	Useful when pages require a browser context or headless collection.
Trivy	Repository vulnerability/secret scanning	Industry-recognized scanner for vulnerabilities, secrets, and misconfigurations.
Gemini / Ollama	NLQ, translation, local intelligence answering	AI assistance for natural-language workflows while keeping fallback behavior available.

8. Why MongoDB Instead of SQL

MongoDB was selected because DarkPulse stores threat intelligence from many sources that do not share one fixed schema. A news record, ransomware leak record, defacement record, GitHub scan, APK result, credential exposure, and confidential-data finding all have different fields. Forcing all of this into SQL tables would require many nullable columns, many join tables, and constant migrations whenever a collector changes.

Requirement	MongoDB Advantage	Why SQL Was Less Suitable
Different source formats	Stores each item as a flexible document.	Requires rigid schemas and frequent migrations.
Raw evidence preservation	Can keep nested raw payloads alongside normalized fields.	Nested data needs multiple related tables or JSON columns with less natural querying.
Fast upserts and dedupe	Uses dedupe keys and document upserts per collector/source.	Possible in SQL, but more complex across many source-specific tables.
Search over mixed text fields	Regex and text search can work across varied document shapes.	Needs carefully designed indexes and schemas for every content type.
Prototype speed	Collectors can evolve without breaking the whole database.	Schema-first development slows experimentation with new OSINT sources.

Conclusion: SQL is excellent for highly structured transactional data, but DarkPulse is an intelligence platform with changing, nested, source-specific evidence. MongoDB better matches that reality.

9. Frontend Choice and Design Rationale

The frontend is built with vanilla HTML, CSS, and JavaScript. This was selected because the project needs a reliable local dashboard without the complexity of a frontend build system. The browser can load the app directly from FastAPI, and the JavaScript can call backend APIs using fetch.

A framework such as React, Angular, or Vue could also be used, but for this project the main challenge is intelligence workflow integration, not component-library complexity. Vanilla JavaScript made the product easier to run, easier to debug during demos, and lighter for local deployment.

- **Why not React?** React is powerful, but it adds package management, compilation, state framework decisions, and deployment complexity.
- **Why not PHP?** PHP is workable for traditional server-rendered sites, but DarkPulse needs async Python collectors, scanner integration, AI calls, and Mongo workflows.
- **Why not Java desktop?** A desktop app would reduce browser accessibility and make web-style dashboards/maps harder to iterate.
- **Why this approach?** FastAPI serves the files, JavaScript handles interaction, and the UI remains simple to demonstrate on localhost.

10. Feature-by-Feature Explanation

10.1 Command Center

The command center is the first operational screen. It summarizes intelligence activity, live hotspots, counts, and key feed data. Its purpose is to give the user a quick understanding of what is happening without opening every module.

10.2 Scan Now / One-Click Intelligence Update

The **Scan Now** function triggers `/api/trigger-smart-update`. It creates a job ID, acquires a scan lock, records a run document in MongoDB, stores a dashboard notification, and starts a background intelligence update task. The scan runs in a fast, headless, incremental mode so previous records remain visible while new unique records are added.

- Sources include news, ransomware leaks, social monitoring, and defacement tracking.
- The system avoids starting duplicate scans by using a lock.
- Users can stop a running scan through the stop endpoint.
- Scan results are stored as source-level summaries and new-record counts.

10.3 Live Feed

The live feed combines news, leak, defacement, exploit, social, and API records. It normalizes titles, descriptions, URLs, source labels, dates, entities, country hints, screenshots, evidence links, and AI-style summaries. This makes mixed source data readable in a consistent dashboard format.

10.4 News Intelligence

News collectors gather cybersecurity reporting and advisories. DarkPulse stores processed news in `articles` and can also read raw `news_items`. Each article is converted into a public feed item with title, summary, author/source context, date, tags, and search text.

10.5 Leak Intelligence

Leak collectors track ransomware and disclosure sources. Records can contain victim names, leak dates, content snippets, teams, country information, screenshots, and links. The dashboard helps analysts quickly identify organizations and topics that require attention.

10.6 Defacement Tracking

Defacement modules collect website defacement records, mirror links, attackers, teams, affected websites, servers, countries, and screenshots. This is useful for monitoring hacktivism, public website compromise, and reputational risk signals.

10.7 Exploit Feed

Exploit collectors gather vulnerability and proof-of-concept information from exploit sources. Normalized fields include exploit title, URL, source, content, actor/source hints, code snippets where available, and dates.

10.8 Social and Forum Monitoring

Social modules collect forum/channel intelligence such as sender names, message links, content, platform, message dates, shares, likes, comments, and hashes. This supports early detection of conversations around leaks, tools, or campaigns.

10.9 API and Tool Outputs

API modules handle special checks such as GitHub scanning, APK mirror checks, PC game sources, PakDB-style authorized lookup integration, and SEO analysis. Their outputs are stored in MongoDB and rendered in dedicated UI screens.

10.10 Semantic Search

The semantic search endpoint `/search/semantic` accepts a user query, searches news and threat records, filters and ranks matching feed items, calculates source counts, and suggests the most relevant dashboard view. It is called semantic because it supports broad intelligence discovery across normalized text fields rather than only one exact database column.

Example: a query like “ransomware university leak” can surface leak, news, and social records that mention related terms, then guide the user toward the dominant source type.

10.11 Natural Language Query

The `/search/nlq` endpoint can translate human language into a MongoDB aggregation pipeline using Gemini when configured. If no Gemini key is available, it falls back to a simple keyword-based pipeline. This helps users ask questions like “show high-impact leaks about Microsoft” instead of manually writing database filters.

10.12 AI Mongo Intelligence Answering

The `/api/ai/query` endpoint connects to `ollama_mongo_intelligence.py`. It searches MongoDB documents, flattens mixed schemas into readable context, and uses a local Ollama model to generate a concise answer. It is defensive and bounded: large fields are truncated, binary fields are omitted, and errors are returned cleanly.

10.13 Translation

DarkPulse supports translating dashboard text through `/translate/text`. It uses `deep_translator` first and can fall back to Gemini. A cache prevents repeated translation of the same text. This helps international audiences and non-English operators understand intelligence faster.

10.14 Repository Scan

Repository scanning accepts a GitHub URL, validates repository metadata, runs Trivy, and reports vulnerabilities, exposed secrets, and misconfigurations. It calculates a risk score and grade using severity counts. This feature turns DarkPulse into an active defensive scanner, not only a feed viewer.

10.15 APK / Playstore Scanner

The APK scanner accepts a Google Play Store URL, extracts the Android package ID, resolves the app name, builds mod/crack search queries, checks local intelligence, web search results, and known mirror candidates, then assigns low/medium/high risk based on suspicious keywords and domains.

10.16 PC Game Scan

The PC game scan searches configured PC game/mod sources and returns app name, URL, version, content type, download link, size, latest date, and mod-feature metadata where available. This demonstrates how DarkPulse can adapt the same intelligence model to software piracy/malware-risk monitoring.

10.17 Credential Checker

The credential checker loads local stealer-log JSON/JSONL/NDJSON files from `data/credential_checker`, converts them into normalized MongoDB exposure records, and lets the user search by domain, email, host, or related text. It tracks datasets, counts, dates, source domains, channels, and metadata tags.

10.18 Confidential Data Analysis

Confidential-data analysis accepts local text, CSV, JSON, or log files and detects indicators such as payment-card-like values, CVV patterns, passwords, tokens, emails, phone numbers, and IP addresses. It uses Luhn validation for payment cards and assigns risk levels based on detected type and confidence.

Important evaluation note: the intended security design is to return only safe findings and avoid exposing sensitive values. This workflow must be kept under authorized defensive use, and redaction behavior should be reviewed before any production deployment.

10.19 SEO Checker

The SEO checker analyzes page title, meta description, status code, crawlability, viewport, language, canonical tag, H1 usage, image alt text, and link text. It returns a numeric score, grade A-F, audit findings, and improvement suggestions. It can use Google PageSpeed when configured, otherwise it uses a local fallback audit.

10.20 Healing Monitor

The healing monitor discovers collector scripts, checks live reachability, stores HTML snapshots, compares DOM drift, measures selector health, logs events, and generates safe repair previews. This is a major uniqueness point because scraper-based systems often break when websites change their HTML.

11. AI, Classifiers, Confidence, and Accuracy

DarkPulse uses a combination of rule-based classifiers, external scanner results, confidence labels, and optional AI assistance. The project repository does not contain a completed formal ML benchmark dataset with final precision/recall/accuracy numbers. Therefore, the honest evaluation is based on deterministic validation logic, scanner coverage, and documented confidence levels.

Classifier / Model	Purpose	Accuracy / Confidence Explanation
Feed Type Classifier	Maps records into news, leak, defacement, exploit, social, or API streams.	Rule-based from collector prefixes and source metadata. Accuracy depends on correct collector tagging; deterministic for known prefixes.
Semantic Search Ranking	Finds cross-source records relevant to a natural query.	Uses normalized text matching, filtering, source counts, caching, and ranking by feed sort. No formal ML accuracy percentage is stored.
Confidential Data Detector	Detects payment cards, CVV, passwords, tokens, emails, phones, and IPs.	Regex plus Luhn validation for cards. Payment-card findings are high-confidence when Luhn passes; phone/email/token findings still require analyst verification.
APK Risk Classifier	Labels APK mirror results low, medium, or high risk.	Rule-based from suspicious domains and keywords such as mod apk, cracked apk, premium unlocked, unlimited money, and hack apk.
Repository Risk Grader	Grades GitHub repository security posture.	Tool-backed using Trivy output. Risk score weights CRITICAL, HIGH, MEDIUM, LOW, and SECRETS. Accuracy depends on Trivy's supported target coverage.
SEO Grade Classifier	Maps SEO score to A-F.	Deterministic thresholds: A for 0.90+, B for 0.80+, C for 0.70+, D for 0.50+, F below 0.50.
Healing Drift Classifier	Labels DOM/source health as no/minor/major change and selector health.	Rule-based comparison of snapshots and selector tests. Useful for operational monitoring, not a predictive ML model.
Gemini / Ollama AI	Translation, NLQ, and local intelligence answer generation.	Generative AI output is assistive. It should be treated as analyst support and verified against stored source records.

Recommended evaluation statement for presentation: “DarkPulse currently uses deterministic classifiers and tool-backed scanners rather than a trained ML benchmark. We evaluate correctness through unit tests, controlled sample inputs, source tagging consistency, Trivy output, Luhn validation, and manual analyst verification. Future work includes building a labeled dataset for precision, recall, F1 score, and confusion-matrix reporting.”

12. Security, Ethics, and Privacy Controls

- DarkPulse is designed for authorized defensive intelligence and research.
- Collectors should only be used on legal, approved, public, or locally controlled data sources.
- Configuration is loaded from environment variables to avoid hardcoding secrets.
- Authentication includes login, registration approval, JWT tokens, optional 2FA flows, and admin user management.
- Repository scanning requires a valid GitHub token and avoids exposing credentials in normal UI text.
- Credential and confidential-data workflows must be used only with authorized datasets.
- MongoDB stores operational history so results can be audited and reviewed later.

13. Testing Strategy

Testing is divided into unit tests, integration checks, UI behavior checks, and manual demonstration tests. The repository includes tests around confidential-data analysis behavior and UI safety expectations. Additional manual tests should verify scan workflows, Mongo persistence, source status, feed pagination, search, translation fallback, and scanner histories.

Test Area	What Should Be Verified
Database	Records are upserted with dedupe keys, counts update correctly, and collections match dashboard modules.
Scan Now	Only one scan runs at a time, status changes from queued/running to terminal states, and stop works.
Feed and Search	News/threat records appear, filters work, semantic search returns relevant matches, and suggested view is sensible.
Repository Scan	Valid GitHub URLs scan successfully, invalid/private/token errors are clear, and results are cached.
Credential Checker	Datasets sync, pagination works, and query results match expected domain/email records.
Confidential Analysis	Luhn validation, detection confidence, risk levels, and safe rendering behavior are verified.
Healing Monitor	Targets are discovered, snapshots are stored, selector health is measured, and repair previews are reversible.

14. How to Present DarkPulse to the Target Audience

The best presentation strategy is to show DarkPulse as a real analyst workflow, not only as code. The audience should see the pain point first, then the product solving it live.

1. **Start with the problem:** “Threat data is scattered across news, leaks, exploits, social channels, repositories, and local exposure files.”
2. **Show the command center:** Explain that DarkPulse gives one operational view of cyber signals.
3. **Run Scan Now:** Demonstrate hidden background collection and explain that cached data remains visible.
4. **Open Live Feed:** Filter by news, leaks, defacement, exploit, social, and API streams.
5. **Use semantic search:** Search a natural phrase and show relevant records plus suggested source view.
6. **Run repository scan:** Show vulnerability, secret, misconfiguration, grade, and risk score.
7. **Run credential checker:** Search an authorized local dataset and show structured exposure results.
8. **Show confidential-data analysis:** Explain detection, confidence, risk, and ethical handling.
9. **Open healing monitor:** Highlight that DarkPulse monitors its own collector reliability.
10. **Close with uniqueness:** “This is not a simple dashboard; it is a local intelligence, scanning, search, and operations console.”

Why the audience will be impressed: DarkPulse demonstrates full-stack engineering, security research, MongoDB schema flexibility, AI-supported search, defensive scanning, operational monitoring, and a polished dashboard in one integrated system.

15. Limitations and Future Work

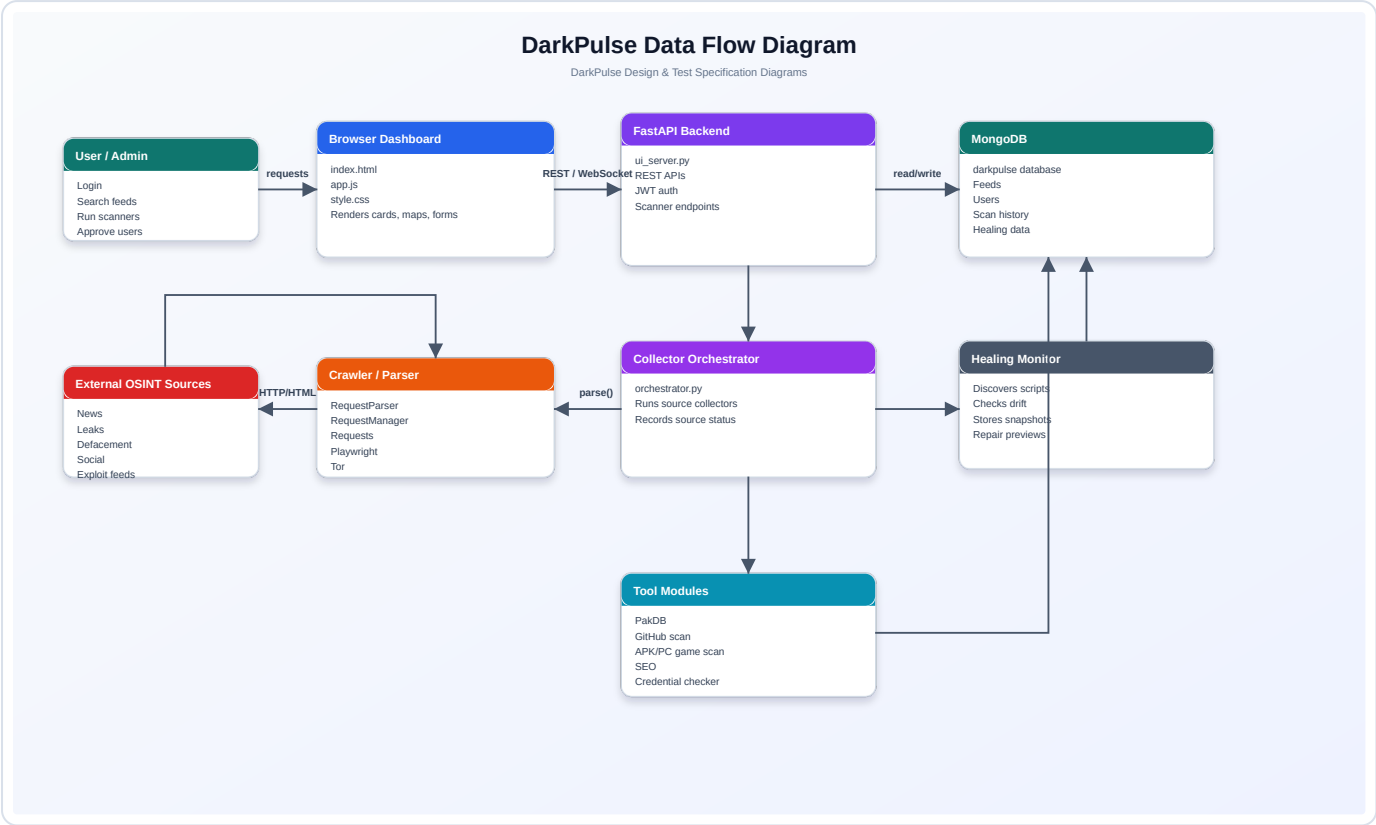
- A formal labeled dataset should be created to calculate precision, recall, F1 score, and accuracy for each classifier.
- Semantic search can be improved with vector embeddings and MongoDB vector search or a dedicated vector database.
- Confidential-data redaction should be reviewed and locked down before any production deployment.
- Role-based access can be expanded for analyst, admin, auditor, and read-only users.
- Collector coverage can be improved with more sources and stricter legal/ethical source allowlists.
- Reports can be exported as PDF/CSV from the dashboard for management-level briefings.
- Automated end-to-end UI tests can be added for every major workflow.

16. Conclusion

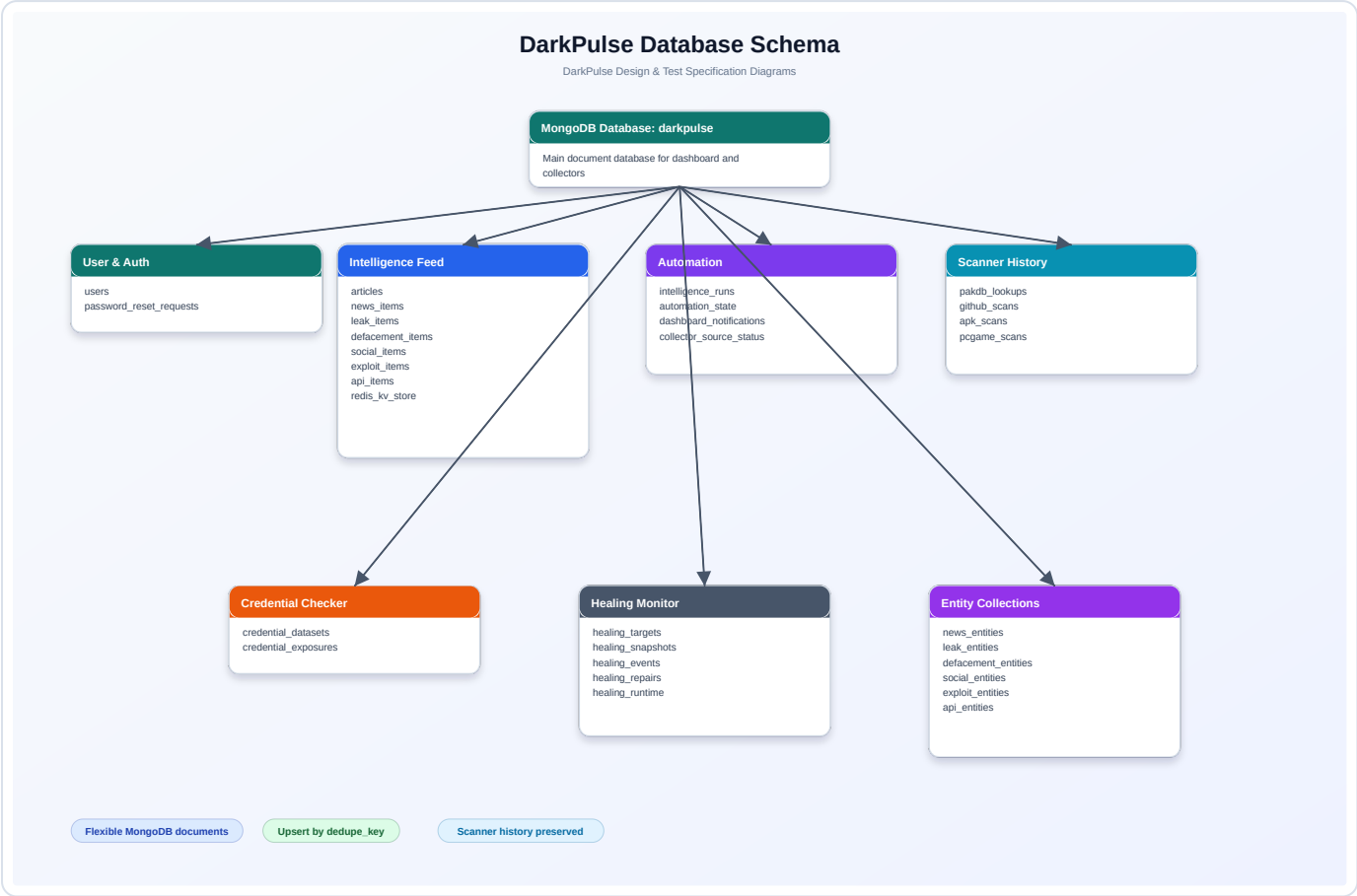
DarkPulse is a strong final-year project because it combines real-world cybersecurity needs with practical software engineering. It includes a professional frontend, FastAPI backend, MongoDB persistence, collector orchestration, multiple intelligence streams, semantic search, AI assistance, vulnerability scanning, exposure checking, and self-healing monitoring.

The product is unique because it does not stop at collecting data. It turns scattered data into usable intelligence, helps users search and explain it, and gives operators tools to keep the collection system reliable. For the target audience, DarkPulse saves time, improves visibility, and creates a more impressive and understandable intelligence workflow.

Appendix A: Architecture Diagram



Appendix B: Database Schema Diagram



Appendix C: Deployment Diagram

